

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ТЕОРИЯ

ЧТО ТАКОЕ CANVAS?

До сих пор мы рисовали интерфейс с помощью готовых компонентов: Text для текста, Button для кнопок, Image для картинок. Но что если нужно нарисовать что-то нестандартное - например, птичку, которая летает по экрану?

Canvas в Jetpack Compose - это 'холст', как в программе для рисования. Мы можем рисовать на нём линии, круги, прямоугольники и даже загружать картинки. Canvas даёт полный контроль над каждым пикселем.

В отличие от обычных компонентов, Canvas не перерисовывается автоматически при изменении данных. Мы должны сами сказать ему: 'Нарисуй птичку вот здесь'. Это делает Canvas идеальным для игр!

ИГРОВОЙ ЦИКЛ (GAME LOOP)

Каждая игра работает по одному принципу: бесконечный цикл, который выполняется десятки раз в секунду. В каждом кадре цикла происходит три вещи:

1. ОБНОВЛЕНИЕ: считаем новые позиции всех объектов (птичка падает, трубы двигаются)
2. ПРОВЕРКА: проверяем столкновения и условия победы/поражения
3. ОТРИСОВКА: рисуем всё на экране с новыми позициями

В нашем Flappy Bird цикл работает примерно 60 раз в секунду (каждые 16 миллисекунд). Это стандартная частота для плавной анимации.

ФИЗИКА В ИГРАХ: ГРАВИТАЦИЯ И СКОРОСТЬ

Гравитация - это сила, которая постоянно тянет птичку вниз. В каждом кадре мы добавляем гравитацию к скорости падения.

Когда игрок нажимает экран, мы задаём отрицательную скорость (вверх). Гравитация постепенно уменьшает эту скорость, и птичка начинает падать обратно.

terminalVelocity - это максимальная скорость падения. Без неё птичка падала бы всё быстрее и быстрее, улетая за пределы экрана.

ПРОБЛЕМА MVC И VIEWMODEL

В больших приложениях код делится на части: View (вид, то что видит пользователь) и Model (данные, логика). ViewModel - это мост между ними.

FlappyBirdViewModel хранит всё состояние игры: позицию птички, список труб, счёт. Экран FlappyBirdScreen только показывает то, что есть в ViewModel.

StateFlow - это специальный тип данных, который 'поток' обновлений. Когда ViewModel меняет состояние, экран автоматически получает новые данные через collectAsState().

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 1: 0. СОЗДАЁМ ФАЙЛЫ (ОЧЕНЬ ВАЖНО!)

```
// В Android Studio создай ДВА файла:  
  
// Файл 1: FlappyBirdViewModel.kt  
// Добавь сюда нужные импорты (Alt+Enter на красном тексте)  
  
// Файл 2: FlappyScreen.kt  
// Добавь сюда нужные импорты (Alt+Enter на красном тексте)
```

ЭТАП 2: 1. Модели данных (В ФАЙЛ ViewModel, внизу)

```
// Перечисление состояний игры  
enum class GameState {  
    NOT_STARTED, PLAYING, GAME_OVER  
}  
  
// Модель трубы: позиция X, центр прохода, ширина и высота прохода  
data class Pipe(  
    val x: Float,  
    val gapY: Float,  
    val width: Float = 160f,  
    val gapHeight: Float = 330f,  
    val passed: Boolean = false  
)  
  
// Полное состояние экрана игры  
data class FlappyState(  
    val gameState: GameState = GameState.NOT_STARTED,  
    val birdY: Float = 800f,  
    val pipes: List<Pipe> = emptyList(),  
    val score: Int = 0,  
    val highScore: Int = 0  
)
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 3: 2. ViewModel: Константы и начальные данные

```
// В ФАЙЛ FlappyBirdViewModel.kt, после импортов:

class FlappyBirdViewModel : ViewModel() {
    companion object {
        // Размеры виртуального игрового поля
        const val BOARD_WIDTH = 1000f
        const val BOARD_HEIGHT = 1600f
        const val GROUND_Y = 1450f

        // Физика птички
        const val BIRD_X = 250f
        const val BIRD_RADIUS = 35f
        private const val FRAME_RATE_MS = 16L
        private const val GRAVITY = 0.7f
        private const val JUMP_VELOCITY = -15f
        private const val TERMINAL_VELOCITY = 18f

        // Трубы
        private const val PIPE_SPEED = 7.5f
        private const val MIN_GAP_Y = 350f
        private const val MAX_GAP_Y = 1100f
    }

    // Поток состояния (экран будет следить за изменениями)
    private val _state = MutableStateFlow(FlappyState())
    val state: StateFlow<FlappyState> = _state.asStateFlow()

    private var gameJob: Job? = null
    private var birdVelocity = 0f
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 4: 3. ViewModel: Нажатие экрана (onTap)

```
// Внутри класса FlappyBirdViewModel:

fun onTap() {
    when (_state.value.gameState) {
        GameState.NOT_STARTED -> startGame()
        GameState.PLAYING -> jump()
        GameState.GAME_OVER -> resetGame()
    }
}

private fun startGame() {
    _state.value = FlappyState(
        gameState = GameState.PLAYING,
        highScore = _state.value.highScore
    )
    birdVelocity = JUMP_VELOCITY
    startGameLoop()
}

private fun jump() {
    birdVelocity = JUMP_VELOCITY
}

private fun resetGame() {
    startGame()
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 5: 4. ViewModel: Игровой цикл и конец игры

```
private fun startGameLoop() {
    gameJob?.cancel()
    gameJob = viewModelScope.launch {
        while (isActive && _state.value.gameState == GameState.PLAYING) {
            updateGame()
            delay(FRAME_RATE_MS)
        }
    }
}

private fun endGame() {
    gameJob?.cancel()
    val finalScore = _state.value.score
    val currentHighScore = _state.value.highScore
    _state.value = _state.value.copy(
        gameState = GameState.GAME_OVER,
        highScore = maxOf(finalScore, currentHighScore)
    )
}

override fun onCleared() {
    super.onCleared()
    gameJob?.cancel()
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 6: 5. ViewModel: Обновление физики и труб

```
private fun updateGame() {
    val currentState = _state.value

    // 1. Гравитация: скорость падения увеличивается каждый кадр
    birdVelocity = (birdVelocity + GRAVITY).coerceAtMost(TERMINAL_VELOCITY)
    val newBirdY = currentState.birdY + birdVelocity

    // 2. Двигаем все трубы влево и убираем ушедшие за экран
    val updatedPipes = currentState.pipes
        .map { it.copy(x = it.x - PIPE_SPEED) }
        .filter { it.x + it.width > 0 }
        .toMutableList()

    // 3. Создаём новую трубу, если последняя уехала достаточно далеко
    val lastPipe = updatedPipes.lastOrNull()
    if (lastPipe == null || lastPipe.x < BOARD_WIDTH - 450f) {
        val randomGapY = Random.nextFloat() * (MAX_GAP_Y - MIN_GAP_Y) + MIN_GAP_Y
        updatedPipes.add(Pipe(x = BOARD_WIDTH, gapY = randomGapY))
    }

    checkCollisions(newBirdY, updatedPipes, currentState)
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 7: 6. ViewModel: Проверка столкновений

```
private fun checkCollisions(newBirdY: Float,
    updatedPipes: MutableList<Pipe>, currentState: FlappyState) {

    var collision = false
    var newScore = currentState.score

    // Границы птички (круг превращаем в квадрат для простоты)
    val birdLeft = BIRD_X - BIRD_RADIUS
    val birdRight = BIRD_X + BIRD_RADIUS
    val birdTop = newBirdY - BIRD_RADIUS
    val birdBottom = newBirdY + BIRD_RADIUS

    // Столкновение с землёй или потолком
    if (birdBottom >= GROUND_Y || birdTop <= 0) collision = true

    // Проверяем каждую трубу
    for (i in updatedPipes.indices) {
        val pipe = updatedPipes[i]
        val gapTop = pipe.gapY - pipe.gapHeight / 2
        val gapBottom = pipe.gapY + pipe.gapHeight / 2

        // Птичка на уровне трубы по горизонтали?
        if (birdRight > pipe.x && birdLeft < pipe.x + pipe.width) {
            // Значит, она должна быть в проходе. Иначе - столкновение!
            if (birdTop < gapTop || birdBottom > gapBottom) {
                collision = true
                break
            }
        }

        // Птичка прошла трубу? +1 к счёту!
        if (!pipe.passed && pipe.x + pipe.width / 2 < BIRD_X) {
            updatedPipes[i] = pipe.copy(passed = true)
            newScore++
        }
    }

    if (collision) endGame() else {
        _state.value = currentState.copy(
            birdY = newBirdY, pipes = updatedPipes, score = newScore
        )
    }
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 8: 7. Экран: Структура и Canvas (Файл FlappyScreen.kt)

```
// В ФАЙЛ FlappyScreen.kt, после импортов:

@Composable
fun FlappyBirdScreen(
    birdPainter: Painter? = null,
    pipePainter: Painter? = null,
    viewModel: FlappyBirdViewModel = viewModel()
) {
    val state by viewModel.state.collectAsState()

    Box(modifier = Modifier.fillMaxSize())
        .background(Color(0xFF70C5CF)) // Небо
        .pointerInput(Unit) { detectTapGestures { viewModel.onTap() } }
    {

        Canvas(modifier = Modifier.fillMaxSize()) {
            val scaleX = size.width / FlappyBirdViewModel.BOARD_WIDTH
            val scaleY = size.height / FlappyBirdViewModel.BOARD_HEIGHT

            withTransform({ scale(scaleX, scaleY, pivot = Offset(0f, 0f)) }) {
                // Здесь будет отрисовка (Шаги 8-9)
            }
        }

        // Здесь будет UI поверх Canvas (Шаг 10)
    }
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 9: 8. Отрисовка: Трубы

```
// Вставляем ВНУТРЬ блока withTransform в Canvas:

// Рисуем каждую трубу из списка
state.pipes.forEach { pipe ->
    val gapTop = pipe.gapY - pipe.gapHeight / 2
    val gapBottom = pipe.gapY + pipe.gapHeight / 2

    if (pipePainter != null) {
        // С картинкой: верхняя труба (перевернута на 180)
        val topPipeSize = Size(pipe.width, gapTop)
        withTransform({
            translate(left = pipe.x, top = 0f)
            rotate(degrees = 180f, pivot = Offset(pipe.width / 2, gapTop / 2))
        }) { with(pipePainter) { draw(size = topPipeSize) } }

        // С картинкой: нижняя труба
        val bottomPipeSize = Size(pipe.width, GROUND_Y - gapBottom)
        translate(left = pipe.x, top = gapBottom) {
            with(pipePainter) { draw(size = bottomPipeSize) }
        }
    } else {
        // Без картинки: прямоугольники
        drawRect(Color(0xFF73C73E), Offset(pipe.x, 0f), Size(pipe.width, gapTop))
        drawRect(Color(0xFF539E1B), Offset(pipe.x, 0f), Size(pipe.width, gapTop), style =
            Stroke(width = 4f))
        drawRect(Color(0xFF73C73E), Offset(pipe.x, gapBottom), Size(pipe.width, GROUND_Y
            - gapBottom))
        drawRect(Color(0xFF539E1B), Offset(pipe.x, gapBottom), Size(pipe.width, GROUND_Y
            - gapBottom), style = Stroke(width = 4f))
    }
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 10: 9. Отрисовка: Земля и Птичка

```
// Всё ещё ВНУТРИ withTransform, после труб:

// 1. Рисуем землю (коричневая основа + зелёная трава)
drawRect(Color(0xFFD2B48C), Offset(0f, GROUND_Y), Size(BOARD_WIDTH, BOARD_HEIGHT - GROUND_Y))
drawRect(Color(0xFF55A630), Offset(0f, GROUND_Y), Size(BOARD_WIDTH, 40f))

// 2. Рисуем птичку
val birdY = state.birdY
val birdX = BIRD_X
val birdRadius = BIRD_RADIUS

if (birdPainter != null) {
    // С картинкой
    translate(left = birdX - birdRadius, top = birdY - birdRadius) {
        with(birdPainter) { draw(size = Size(birdRadius * 2, birdRadius * 2)) }
    }
} else {
    // Без картинки: жёлтый круг + клюв + глаз
    val beakPath = Path().apply {
        moveTo(birdX + birdRadius - 5f, birdY - 10f)
        lineTo(birdX + birdRadius + 18f, birdY)
        lineTo(birdX + birdRadius - 5f, birdY + 10f)
        close()
    }
    drawPath(path = beakPath, color = Color(0xFFFF77F0))

    drawCircle(Color(0xFFFF9C80E), radius = birdRadius, center = Offset(birdX, birdY))
    drawCircle(Color(0xFFD6A000), radius = birdRadius, center = Offset(birdX, birdY),
        style = Stroke(width = 3f))
    drawCircle(Color.White, radius = 8f, center = Offset(birdX + 10f, birdY - 10f))
    drawCircle(Color.Black, radius = 4f, center = Offset(birdX + 12f, birdY - 10f))
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 11: 10. Интерфейс: Счёт и экраны состояний

```
// Внутри Box (после Canvas), накладывается поверх игры:

Column(
  modifier = Modifier.fillMaxSize().padding(32.dp),
  horizontalAlignment = Alignment.CenterHorizontally,
  verticalArrangement = Arrangement.SpaceBetween
) {
  // Счёт вверху экрана
  Text(text = "Score: ${state.score}", fontSize = 36.sp, fontWeight =
    FontWeight.ExtraBold, color = Color.White)

  Box(modifier = Modifier.weight(1f), contentAlignment = Alignment.Center) {
    when (state.gameState) {
      GameState.NOT_STARTED -> {
        Text("TAP TO START", fontSize = 24.sp, fontWeight = FontWeight.Bold, color =
          Color.White)
      }
      GameState.GAME_OVER -> {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
          Text("GAME OVER", fontSize = 44.sp, fontWeight = FontWeight.Black, color =
            Color(0xFFD62828))
          Spacer(modifier = Modifier.height(8.dp))
          Text("High Score: ${state.highScore}", fontSize = 20.sp, fontWeight =
            FontWeight.Bold, color = Color.White)
          Spacer(modifier = Modifier.height(16.dp))
          Text("TAP TO RESTART", fontSize = 18.sp, fontWeight = FontWeight.SemiBold, color
            = Color.White)
        }
      }
      GameState.PLAYING -> {}
    }
  }
}
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ЭТАП 12: 11. Как запустить игру (MainActivity.kt)

```
// В MainActivity.kt замени содержимое на:

@Composable
fun MainApp() {
    Surface(color = MaterialTheme.colorScheme.background) {
        FlappyBirdScreen()
    }
}

// Если есть картинки для птички и труб:
// Положи файлы me.png и lacy_promo.png в папку res/drawable/

@Composable
fun MainApp() {
    Surface(color = MaterialTheme.colorScheme.background) {
        FlappyBirdScreen(
            birdPainter = painterResource(R.drawable.me),
            pipePainter = painterResource(R.drawable.lacy_promo)
        )
    }
}

// В onCreate() MainActivity:
// setContent { MainApp() }
```

ЭТАП 13: 12. Предпросмотр (Preview)

```
// Пишется в ФАЙЛЕ FlappyScreen.kt, СНАРУЖИ функции FlappyBirdScreen:

@Preview
@Composable
fun FlappyBirdPreview() {
    MaterialTheme {
        FlappyBirdScreen()
    }
}

// ПРИМЕЧАНИЕ: Preview покажет начальное состояние (NOT_STARTED).
// Чтобы увидеть игру в действии - запускай на эмуляторе или телефоне!
```

СОЗДАЕМ СВОЙ FLAPPY BIRD

МОДУЛЬ: ИГРОВАЯ ЛОГИКА И CANVAS | ID: 7

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Настрой сложность: измени гравитацию (GRAVITY) и скорость труб (PIPE_SPEED). Попробуй сделать так, чтобы проход между трубами становился меньше при счете больше 10!